

ZT scanning-DIA → develop: staged re-implementation plan

Pioneer.jl — branch `feat/zt-scanning-v2`

2026-09-09

Branch `feat/zt-scanning-v2`, cut clean from `origin/develop` @ 9baafa921. Source of truth for prior work: `origin/feat/zt-batched-collapse` @ 7653d38d0 (45 commits, 375 behind develop). Handoff: `/Volumes/d.goldfarb/.../ZT_HANDOFF_2026-09-09/`.

1. Principles

1. **Re-implement conceptually, don't replay commits.** The old branch is a research trail with reversals in it. We port the *conclusions*, not the path.
2. **Every stage ends at a number.** Each stage has an `A_REP1` precursor-ID checkpoint. If the number is wrong, stop — do not stack the next stage on an unverified one.
3. **Dead ends are not ported at all** (see Section 10).
4. **Known-bad designs get fixed at build time**, not ported-then-optimized later.
5. **ZT activation moves from env var to config.** The single most expensive bug in the whole project was a silent no-op; the new design makes it structurally impossible.

2. Verification ladder

stage	what lands	A_REP1 target	actual
0	nothing (control)	measure (old ZT-off 23,013)	22,656 ✓
1	geometry + activation	unchanged from stage 0	22,656 bit-ident. ✓
2	expansion + collapse	~26,552	—
3	bitvec window alignment	~27,234	—
4	shape / elution features	≥ 27,234	—
5	wide-emit tol=2	~30,359	—
<i>reference: DIA-NN direct</i>		<i>30,478</i>	—

All runs: `A_REP1` single file, `--threads 12 --gcthreads 8,1`, MBR off, ~170s/file.

3. Stage 0 — develop baseline (no code changes)

Run A_REP1 on clean develop with no ZT anything. Develop has moved 375 commits since the ZT branch forked, so the old 23,013 control is not assumed to still hold. **This number is the control for every stage after it.**

- Deliverable: `results/s0_develop_baseline`, ID count recorded.
- Also confirms the environment: threads, memory, library path, arrow integrity.

4. Stage 1 — Geometry and activation (DONE)

Concept: teach the search that this acquisition is a *swept* quadrupole — the recorded ~ 1.02 Da `isolationWidthMz` is only the Q1 step, while the physical window is several Da swept across m/z . Q1 bins are contiguous MS2 scans within a cycle.

Detection, storage and logging **only**. No consumer reads the geometry, so results are unchanged.

- ZTGeometry (`src/Routines/SearchDIA/zt_geometry.jl`): `bin_step`, `nominal_width`, `bins_per_ramp`, `metascan_k`.
- **Config-driven activation:** `acquisition.scanning_quad + acquisition.metascan_k`, not `PIONEER.ZT_METASCAN_K`. `acquisition` parses generically to a `NamedTuple`, so no parser change was needed.
- **Lazy, per file**, from `execute_search`'s existing loop — measures the already-open spectra, one file at a time, no extra file opens. Idempotent via cache.
- Every file logs its status **exactly once**, ON or OFF. Silence is never a valid state; the cache stores nothing for “checked, not ZT”.

★ `bin_step` must come from `centerMz` differences, **NOT** `isolationWidthMz`

The recorded width is **dithered 50/50 between two quantized values** that strictly alternate (1.02, 1.0241, 1.02, 1.0241, ...): 1.0199999809 at 50.04% and 1.0241279602 at 49.96%, separated by 4.13×10^{-3} Da = **34,628 Float32 ulps** — far too large to be a representation artifact. Their *average* is the true step, so:

estimator		value
<code>median(isolationWidthMz)</code>	1.0200	(degenerate on a 50/50 split)
<code>mean(isolationWidthMz)</code>		1.0220625
<code>median(diff(centerMz))</code>	1.0220642	(direct, robust)

The median of `isolationWidthMz` is off by 2.1×10^{-3} Da, compounding to **a full bin of drift across a 496-bin ramp** ($495 \times 1.0200 = 504.90$ vs the actual 505.92 Da). The old branch's `_zt_bin_step` used exactly that median.

`nominal_width` keeps the dithered median but is used *only* for the tiling check, where the 2% tolerance absorbs the 0.2% discrepancy.

Measured lattice (reference data)

A_REP1 bin_step=1.0220642 nominal=1.02 bins/ramp=496 tiles=true span(k=6)=13.29 Da; B_REP1 1.0220947; C_REP1 1.0222473.

Within a file the lattice is exact — ramp-start std **0.0** and step std $\sim 3 \times 10^{-8}$ over 818 cycles — so 8 sampled cycles reproduce the full-file value. Between files it genuinely drifts (ramp start by ~ 60 mDa, step by 1.8×10^{-4} Da), which is why the lattice is stored **per file** while `metascan_k`, a config choice, is global.

Cycle = return to the same `centerMz` (`compute_cycle_idx`s), **not** per-MS1; MS1 scans inherit the current cycle index. On this data the two coincide — exactly one MS1 at the head of each ramp.

Checkpoint (met): 22,656 — identical precursor set *and* identical q-value vector to stage 0, max $|\Delta q| = 0.0$. Gotcha #1 is now structurally impossible.

5. Stage 2 — Metascan expansion + collapse

Concept: a precursor’s fragments are smeared across $\pm k$ bins, so (a) give it candidacy in all $\pm k$ bins and (b) collapse the resulting per-bin PSMs back to one meta-PSM per (cycle, precursor) carrying a weight profile.

- **Two-width quad model:** narrow box for the fragment index, wide box for deconvolution. (Moved here from stage 1 — a wide deconv box changes results immediately, so it cannot sit in a stage whose checkpoint is “no change”. It only becomes meaningful once candidates span bins.)
- `expand_to_metascans!` — union candidates over $\pm k$ same-cycle MS2 bins. **Re-implement parallel** with per-scan thread-local buffers; the original allocates a `Vector{Set{UInt32}}` over every searched scan.
- `collapse_to_metascans` — the substantive core (~ 595 lines). Port structurally; **drop the batched and disk-spill variants** for now (single code path, easier to verify).
- `filter_to_center_bin!` — center-bin candidacy, i.e. pre-wide-emit behavior.
- Skip spectral scoring for non-center bins (perf; no ID effect).

Checkpoint: $\sim 26,552$.

6. Stage 3 — Bitvec calibration window alignment

Concept: the bitvec LUT must be calibrated with the same precursor window candidacy actually uses. On ZT these were mismatched — calibration used the QuadTuning-tuned model while candidacy used the narrow square box. Correctness fix, not a tuning knob.

Checkpoint: $\sim 27,234$ (the alignment alone was worth +682).

Note: `PIONEER_BITVEC_MIN_EXCESS` (θ) is *not* the chosen mechanism — wide-emit supersedes lowering θ . Add the knob only as a diagnostic.

7. Stage 4 — Shape / elution features

Within-metascan fragment shape, across-cycle elution, metascan-aware chromatogram integration; `zt_tri_cosine`, `zt_entropy`, `zt_tri_pcor` promoted to the 2nd-pass model.

This is the hot conflict zone. Develop churned `IntegrateChromatogramsSearch/Utils.jl` by ~ 1020 lines and `IntegrateChromatogramsSearch.jl` by ~ 464 while ZT touched them by 88 and 46. Do **not** port the old diff. Re-derive against develop’s current implementation — note that `cf30cc4bd` (“run develop chromatogram features post-collapse; drop bespoke elution impl”) already moved in exactly that direction, so the honest delta here is small.

Checkpoint: $\geq$ stage 3.

8. Stage 5 — Wide-emit candidacy (the winner)

Concept: today a precursor is emitted only if its *center* bin clears the bitvec. Give it an emission chance in *every* bin, then re-anchor each emission to the center bin. It survives if it cleared in **any** bin. Different bins expose different fragment subsets, so these are real second looks, not noise admission.

- Widen the fragment-index candidacy box, `tol = 2.0 Da` half-width. (`tol = 4.0` saturates and collapses to 23,013 — there is a sweet spot; do not overshoot.)
- `map_any_hit_to_center!` — **re-implement, do not port.** The original is a serial `Dict{Int,Set{UInt32}}` plus a `Set{(cycle,prec)}` dedup over ~ 60 M emissions: 181 s, the #2 cost in the whole search and the top serial bottleneck. Replace with per-cycle bucket arrays and direct nearest-bin arithmetic from the known uniform bin step, instead of the $\pm$ halfbins linear scan. This is the handoff’s next-step #2, done at build time.

Checkpoint: $\sim 30,359 = \text{DIA-NN parity}$. This is the headline gate.

9. Stage 6 — Tractability

- **metascan.k sweep** — a speed/recall tradeoff, **not** a correctness fix. Size k against the *measured* effective transmission (Gaussian, $\text{FWHM} \approx 6.7 \text{ Da}$), not the nominal 5 Da: $6.7/1.0221 = 6.55$ bins FWHM, so $\sigma = 2.78$ bins.

k	span	in σ	$\sim$ fraction of signal
3	$\pm 3.07 \text{ Da}$	$\pm 1.08\sigma$	$\sim 72\%$
4	$\pm 4.09 \text{ Da}$	$\pm 1.44\sigma$	$\sim 85\%$
5	$\pm 5.11 \text{ Da}$	$\pm 1.80\sigma$	$\sim 93\%$
6	$\pm 6.13 \text{ Da}$	$\pm 2.16\sigma$	$\sim 97\%$

$k = 6$ is well chosen, not an overshoot — an earlier draft claimed $k = 3$ was “more physically correct” by comparing the nominal 5 Da full-width against a half-width. That was wrong. $k = 3$ discards $\sim 28\%$ of the transmitted signal; it may still win on net because the tails are also where

cross-talk lives and deconv cost scales with bin count, but that is an empirical question. $k = 4$ and $k = 5$ are the more interesting probes.

- Split deconv timing: build (`run_fused!`) vs solve (`solvePoissonMM_fast!`).
- Disk-spill only if memory demands it — this machine has 48 GB and the old peak was ~ 22 GB, so likely unnecessary.

10. Explicitly out of scope (do not port)

dropped		why
no-reset (<code>PIONEER_ZT_MERGE_K</code>)	merge	abandoned — 29,485 < wide-emit 30,359, and <i>not</i> cheaper (463 M vs 519 M candidates)
merged-LUT calibration		over-prunes (15/256 patterns pass), 24,292 — worse than baseline
empirical Gaussian quad transmission		tested <i>worse</i> (24,583); square box stays the default. Drops <code>zt_transmission_estimate.jl</code> + <code>empiricalQuadModel.jl</code> entirely
batched (<code>PIONEER_ZT_BATCHED</code>)	collapse	rejected — not byte-identical, slower
diagnostic dump knobs		re-add individually when a specific question needs one

Roughly 40% of the old branch’s surface area, removed before we start.

11. Known conflict map (ZT lines vs develop lines since fork)

Clean — develop never touched ($\sim 1,150$ lines, near-zero risk): `metascan_collapse.jl` (595, new) · `QuadTuningSearch.jl` (75) · `process_scans_fused.jl` (61) · `PartitionedFragmentIndex/search.jl` (55) · `BitVecCalibration.jl` (41) · `getFragIsotopes.jl` (41) · `ParameterTuningSearch.jl` (33) · `PrecEstimation.jl` (12)

Hot — real merge work:

file	ZT	develop
<code>IntegrateChromatogramsSearch/utils.jl</code>	88	1020
<code>IntegrateChromatogramsSearch.jl</code>	46	464
<code>score_psms.jl</code>	17	303
<code>spectralDistanceMetrics.jl</code>	35	131
<code>MainSearch.jl</code>	225	108
<code>LibrarySearch.jl</code>	372	72

Warning: `score_psms.jl` also carries uncommitted work on `feat/empirical-library-isotopes` in the main checkout — coordinate before touching it.

12. Standing gotchas

- Always pass `--threads` (bare `julia` is `Threads:1`, $\sim 7\times$ slower). Verify via the `Threads:` line in `pioneer_search.debug.log`.
- `main_search_psms/1.arrow q_value` is an all-zeros placeholder. Use `trace_prob`; final FDR is `precursors_long.qval`.
- Use the peptidoform-aware join (`overlap-pepform.jl`) for DIA-NN comparisons.
- Never run two searches concurrently when timing anything.

13. The strategic context

Wide-emit reaching parity is *not* the end state. The decomposition of the 6,352 DIA-NN-only precursors says **78% (4,898) are already extracted at the correct retention time** — 98.2% within 0.1 min of DIA-NN, median $|\Delta RT| = 0.000$ — and simply rank below the 1% FDR bar (median `trace_prob` 0.285 vs a passing floor of 0.982). Only 22% is a genuine recall gap.

So this port restores the extraction capability; the remaining gap is a **scoring** problem. That is also what `feat/empirical-library-isotopes` is already working on in the main checkout.